HiFI: A Hierarchical Framework for Incremental Learning using Deep Feature Representation

Ankita Raj, Anima Majumder and Swagat Kumar

***Abstract*— The presented work focuses on automatic recognition of object classes while ensuring near real-time training required for recognizing a new object not seen previously. This is achieved by proposing a two-stage hierarchical deep learning framework which first learns object categories using a Nearest Class Mean (NCM) classifier applied directly to CNN features and then, uses a two-layer artificial neural network to learn the object labels within each category. In order to recognize a new object not seen earlier, the category is identified first and then the second stage neural network is incrementally trained with the features of the new object without forgetting previously learnt labels. The proposed hierarchical framework is shown to provide comparable recognition accuracy with significant reduction in overall computational time in recognizing new objects compared to methods that use end-to-end re-training. The efficacy of the approach is demonstrated through comparison with existing state-of-the-art methods on the publicly available CORe50 dataset.**

I. Introduction

An e-commerce retail warehouse has to deal with millions of products with several thousands of new products getting added every day. Currently prevalent barcode based system to log and retrieve product data in and out of the warehouse management system requires manual intervention at several stages - putting the barcode level, locating barcode location on the product and getting it scanned through appropriate positioning etc. The recent success of deep learning based object recognition systems [1] [2] [3] [4] may soon remove these human interventions by recognizing products directly from images obtained using cameras mounted at various locations to monitor the product movement within the warehouse. However, the performance of these deep learning methods rely on time consuming and computationally intensive training phases requiring huge amount of labelled datasets which are usually created manually through expensive human efforts [5] [6]. In such a scenario, one pertinent question to ask would be "How to deal with new products that are being introduced into the warehouse system everyday?" The naive approach of re-training the whole network with new as well as old data would become practically untenable in a real world scenario. Researchers have tried to solve this problem using methods such as continuous learning or incremental learning techniques [7] [8] [9] [10] [11] where a pre-trained deep network is trained with the new dataset while ensuring the old learning is not forgotten. Similarly, there are hierarchical approaches [12] [13] that distribute learning over multiple stages. These hierarchical approaches have been shown to have lower prediction times for large number of classes compared to methods that adopt flat classification. Many researchers are exploring methods like meta-learning [14] [15] [16] and transfer learning [17] [18] to deal with this problem in cases where the number of object classes could be very large. This is otherwise known as the extreme classification problem in the literature.

We attempt to solve this problem by proposing a two-stage hierarchical approach where the object *category* is identified in the first stage and then, a smaller classifier is used to recognize the product label in the second stage. This is based on the intuition that it is much easier to identify a product category like 'coffee' and 'milk' than a particular coffee or a milk brand. The products within a given *category* share common attributes which are easy to spot even from a distance when one can read the labels printed on them. These categories usually do not change with new products and can be learnt off-line with the existing dataset. Once the category is identified, a smaller network could be trained to identify all product labels within this category. Whenever a new item is encountered, its category is recognized in the first stage and then, the second stage is incrementally trained to recognize the new item. As shown later, this approach is much faster than the case where a single network is retrained to learn object labels directly from its features.

The first stage of the framework uses a *nearest class mean (NCM)* [14] [19] based classifier to generate category labels directly from product features obtained from a deep network like CNN [1]. The classifier computes and stores the mean feature representation for each category. The category labels are generated only once using the existing product dataset and is used for identifying the category label for a new object not previously seen by the classifier. The second stage of the framework uses a two layer artificial neural network for each category to learn the mapping between the product (deep) features and their corresponding class labels. Since the number of products within a category will be much smaller compared to the total number of products, the training of these individual networks can be carried out much faster once the category label is identified. In order to recognize a new product not previously seen, the network is incrementally trained using the features of this new product while avoiding catastrophic forgetting by using the concept of protoset memory [8] where the network is retrained using a subset of the complete feature set selected dynamically over time. The resulting framework is shown to provide near real-

[1]Ankita Raj is a PhD student at IIT Delhi. This work was done while she was working at TCS Innovation Labs. Anima Majumder and Swagat Kumar are researchers at TCS Innovation Labs, India. raj.ankita92@gmail.com, anima.majumder@tcs.com, swagat.kumar@tcs.com

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.

time training phase to recognize the new products without compromising on the recognition accuracy. The efficacy of the approach is demonstrated through comparison with other state-of-the-art methods using publicly available CORe50 [20] dataset which is widely used in Continual or Lifelong learning literature as it is provides a large number of images along with their product and category labels.

In short, our major contribution lies in proposing a novel two-stage hierarchical deep learning framework that can incrementally learn the labels of a new product by using a near real-time training phase making it one of the fastest incremental learning approaches in the literature. To be specific, the proposed approach achieves classification accuracies comparable to the state-of-the-art iCaRL method [8] while requiring 20x lesser time for training on incremental batches of data. We believe that the use of category label to reduce the overall training time will have far reaching impact in the extreme classification problem where the number of categories to be generated will be at least one or two order smaller than the actual number of classes to be learnt.

The rest of this paper is organized as follows. A brief overview of related work in this field is provided in the next section. The details of the proposed approach are provided in Section III. Performance analysis of the proposed approach is discussed in Section IV. The conclusion and direction for future work is provided in Section V.

II. Related Work

Being able to incrementally train convolutional networks for new classes is a major open problem in the field of neural networks. When data for new classes becomes available, a network should be able to: *(i)* provide satisfactory results on new classes, *(ii)* retain performance on old classes, and *(iii)* be able to do so with bounded computational and memory requirements [8]. We first describe some of the classical ways in which this problem can be approached, which unfortunately do not work well in the incremental scenario. Then we describe methods developed specifically for incremental learning.

A. Baseline methods

A naive approach to incorporate data for new classes in an already trained model is to continue Stochastic Gradient Descent using the incoming data. We call this method *Continue-SGD* after [20]. This, however, leads to huge drop in accuracy for the previously learnt classes, as the weights of the network adapt to new classes, losing the previously learnt information. This phenomenon is referred to as catastrophic forgetting. At the other extreme, one could retrain the network from scratch using the entire training data encountered so far, whenever new classes become available, but this would have large computational as well as memory requirements. We call this method *Cumulative retraining*, and it can be seen as an upper bound to the expected performance of an incremental method.

B. Incremental methods

Recently, incremental learning has been addressed in several works. [21] provides a comprehensive overview of many of these continual learning methods. There is a class of methods that alleviates catastrophic forgetting on the previously learnt classes by preventing network weights from deviating much from their original values, by using a small learning rate or by regularizing the weights. In *Learning without Forgetting* [7], new parameters are added for new classes, and weights are updated while imposing constraints to preserve the network's output on the previous classes. In [22], learning is slowed down for important parameters thus constraining them to stay close to their original values.

Another relevant approach is where the network's resources are dynamically increased in order to accommodate new tasks. In *Progressive Neural Networks* [9], a pool of networks is maintained, one for each incremental task. Whenever new classes are introduced, a new network is learned parallel to the existing network along with lateral connections between the two networks. It has the drawback that the number of parameters rapidly increases as new classes are introduced.

Another class of methods is based on the complementary learning systems (CLS) theory inspired by the mammalian brain where neocortical system stores acquired knowledge for long-term retention, while the hippocampus rapidly adapts to new information [10] [8] [20]. Of these methods, *iCaRL* is directly relevant to this paper. iCaRL [8] is a trainable feature extractor followed by a single classification layer, and maintains a fixed-size exemplar set consisting of training data from each class observed so far. When new classes are encountered, the exemplar set is updated and classification is done by nearest mean-of-exemplars rule. However the feature representation requires retraining for new batches of data, which make the training times for incremental batches considerably high.

III. Proposed Approach

We propose HiFI, a hierarchical framework for incremental object classification. This section describes HiFI's overall architecture and key components.

A. Architecture

We break the problem of classification of objects into two stages – identification of high-level categories in the first stage, followed by final class prediction within each category in the second stage. For instance, different types of books can be clubbed together under the common category *books*, shampoo bottles of various brands can be grouped into a category called *shampoo*, and so on. The first stage is a high-level classifier which predicts the category that a given object belongs to, i.e. book, shampoo, etc. The second stage comprises of an ensemble of classifiers, one for each category. Given the category prediction from the first stage, the corresponding classifier in the second stage predicts the final class label within that category. For example, the *books*-classifier could have class labels like *Ulysses*, *Macbeth* and

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.

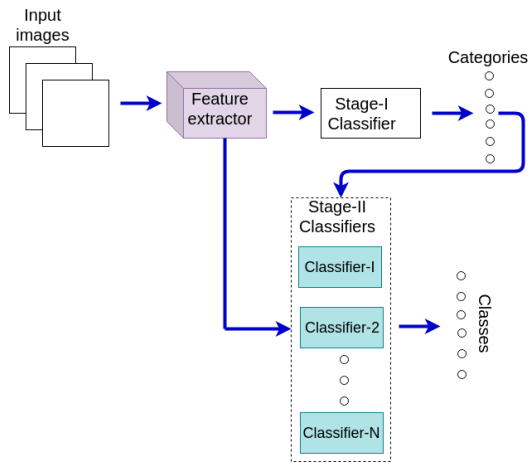

Fig. 1. An architectural overview of HiFI: Given an input image, a feature extractor generates a representation for the image, which is first fed to the Stage-1 classifier to produce a high-level category label. The same feature representation is then fed to one of the classifiers in Stage-2 (corresponding to the predicted category), which gives the final object label.

Jane Eyre; similarly the *shampoo*-classifier could predict labels like *Dove* and *Sunsilk*.

B. The Hierarchy

A vital part of HiFI's two-stage classification approach is defining the hierarchy of classes, i.e. deciding which classes should be grouped together to form higher-level categories. While these categories can be machine-defined or created by a human, for this work, we assume that the dataset contains an inherent grouping of classes.

Consider an initial training batch comprising of L_0 classes and a set of N_0 training points $\{(\boldsymbol{x_i}, y_i)_{i=1}^{N_0}\}$ with D dimensional input images $\boldsymbol{x_i}$ and class labels $y_i \in [1, L_0]$. In the subsequent incremental batches, training instances with new class labels are introduced, so that the number of class labels increases to $(L_0 + L_1)$, $(L_0 + L_1 + L_2)$ and so on for L_1 and L_2 new classes introduced in the second and third batches respectively. Assuming there are a total of M incremental batches (excluding the initial batch), the total number of classes is $L_{tot} = L_0 + L_1 + \cdots + L_M$.

A set of categories $\mathcal{C}$ is defined such that each class belongs to a unique category. A function g maps each class label $l \in [1, L_{tot}]$ to a category $c = g(l) \in \mathcal{C}$. A key requirement for the proposed approach is that all the categories should be known at the onset of training, i.e. at least one object belonging to each category should be presented to the network in the initial training batch.

C. Stage-1 classifier

The first stage classifies objects into high-level categories. The intuition behind splitting the network into 2 stages is that the first stage models a generic grouping of objects in a given dataset, which remains unchanged when new objects are incrementally added to the dataset. Hence the first stage classifier should not have to be retrained every time new object classes are introduced.

We use a *nearest class mean (NCM)* classifier for category-level classification. The NCM classifier computes and stores the mean feature representation per class, and classifies unseen instances with the label of the nearest class mean. Training involves the following steps:

1) **Feature representation learning:** Using the initial batch of training data, a pre-trained classification network is fine-tuned for predicting the final class labels, and the last hidden (bottleneck) layer of this network is used as a feature extractor. This can be interpreted as a non-linear function f that maps each D dimensional input image $\boldsymbol{x_i}$ to a D' dimensional feature vector $\boldsymbol{z_i}$, such that $D' < D$:

$$\boldsymbol{z_i} = f(\boldsymbol{x_i}) \tag{1}$$

The mapping f is a fixed feature representation that does not require re-training for subsequent batches.

2) **Computation of class means:** Using the feature representation learned in the previous step, features are extracted for all training instances. For each class l in the training batch, class mean $\boldsymbol{\mu_l}$ is computed by averaging over all instances belonging to that class.

$$\boldsymbol{\mu_l} = \frac{1}{n_l} \sum_{i:y_i=l} \boldsymbol{z_i} \tag{2}$$

where n_l denotes the number of training instances in the batch belonging to class l.

3) **Protoset creation:** In order to mitigate catastrophic forgetting without having to store the entire training data for previous batches, proposed method uses the concept of protoset memory. For each encountered class l, a set of training examples $\mathcal{P}^{(l)}$ is dynamically selected and stored as the protoset for that class. Following [8], this selection is done using *herding*, whereby training samples closest to the class mean are chosen. The number of protosets per class can be fixed beforehand, and can easily be chosen to be a few orders of magnitude smaller that the actual number of training samples per class.

Note that the computationally intensive step (1) is performed only for the initial batch of training data, while steps (2)-(3) are performed for each subsequent batch that arrives incrementally. This makes the training for incremental batches extremely fast as it only involves a forward pass through the deep convolutional feature extractor, followed by computations on the extracted features.

In the prediction phase, NCM computes the feature representation of the test image $\boldsymbol{x_{test}}$ and assigns the category with the closest class mean. Since the stored mean vectors $\{\boldsymbol{\mu_l}\}_{l=1}^{L_{tot}}$ are indexed by class, NCM first computes the nearest class mean $\hat{l}$, and then maps it to the corresponding category label $\hat{c}$.

$$\hat{l} = \underset{l \in [1, L_{tot}]}{\arg\min} (\boldsymbol{x_{test}} - \boldsymbol{\mu_l}) \tag{3}$$

$$\hat{c} = g(\hat{l}) \tag{4}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.

Fig. 2. Objects in the CORe50 dataset grouped into 10 categories. HiFI leverages this inherent hierarchy in the dataset by first recognizing the category (e.g. *Ball*) of a given image, and then predicting the final class (e.g. *Green ball*).

D. Stage-2 classifiers

Given the category labels from first stage, final class labels are predicted in the second stage. For each category $c \in \mathcal{C}$, an independent classifier κ_c is trained that learns to predict class labels for objects within that category. The classifier κ_c takes as input the feature vector $\boldsymbol{z_i}$ and outputs a class label $\hat{y}_i$ from among the labels belonging to category c.

In order to keep the training time small, simple models are used in this stage which can be trained quickly. The classifier used for each category is a simple 2-layer neural network with 100 units in the hidden layer.

Each individual classifier in the second stage is trained using the samples stored in memory as the protosets for classes. When an incremental batch of data becomes available, Stage-1 first extracts the feature representation from the images and generates protosets for each newly encountered class. Only those classifiers in Stage-2 are then retrained which correspond to the categories to which the new classes are mapped by the first stage; the remaining classifiers are left untouched. Training data for classifier κ_c comprises of the protosets for all classes encountered so far in the training process that belong to category c, which is the set $\{\mathcal{P}^{(l)} | g(l) = c\}$. Even though the protoset size is small, we observed that it is sufficient to train the simple classifiers, as can be verified in Sec IV-D.

IV. Experimental Results and Discussion

Our experiments are designed to evaluate whether the proposed method is effective for learning new object classes in near-real time, while retaining performance on previously learned classes.

A. Dataset

We use the CORe50 dataset [20] for evaluation of the proposed method. This dataset, which consists of images of hand-held objects being moved aroung by a person, has been designed specifically for continuous object recognition. It comprises of 50 household objects, and has been collected in 11 distinct sessions with different backgrounds and lighting. For the incremental learning benchmark, data is presented in 9 sequential batches — the first batch comprises of 10 objects; each new batch presents 5 new objects to be recognized. For each sequential batch, images from 8 sessions are used for training, while the remaining 3 sessions are kept for testing. After each batch, the resulting model is evaluated on a test set comprising images from all the classes seen so far, so that performance on previous as well as newly learnt classes can be measured.

B. Experimental setup

The work in [20] mentions three different continuous learning scenarios - New Instances (NI), New Classes (NC) and New Instances and Classes (NIC). We have used the NC scenario for our experiments, since it best represents the use-case in a warehouse where new training batches comprise of images belonging to new classes not known earlier. We have used the same sequence of classes as described in the code accompanying the CORe50 dataset for the NC scenario.

For the hierarchical grouping in HiFI, the inherent category information of the CORe50 dataset is used. The 50 objects in the dataset can be grouped into 10 distinct categories [20] - *plug adapters*, *mobile phones*, *scissors*, *light bulbs*, *cans*, *glasses*, *balls*, *markers*, *cups* and *remote controls*. Fig-

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.

2 shows the objects grouped into their respective categories. The first batch in the training sequence comprises of training patterns belonging to one class from each category. In each subsequent batch, new training patterns belonging to 5 new classes become available; each of these new classes can be mapped to an already known category.

We compare the proposed approach with *Cumulative retraining* and *Continue-SGD* among the baseline methods, and *NCM* and *iCaRL* from the incremental methods (refer Sec-II). We compare with iCaRL, because it has better performance than other leading methods like Learning Without Forgetting [8]. NCM, though reported to have lower accuracies than iCaRL as per the experiments conducted in [8], has very low training times, and is hence a suitable candidate for comparison with the proposed method.

C. Implementation details

For the deep feature extractor in the first stage of the proposed model, we use a Resnet-50 model pre-trained on the Imagenet dataset. This model is fine-tuned on the initial batch of training data for 10 epochs. Learning rate starts at 0.001 and decays exponentially with a decay rate of 0.8 after every epoch. The network is trained on a 8 GB Quadro M5000M GPU using a mini-batch size of 32. For the protoset memory, we store 100 training samples per class.

For cumulative-retraining, the same learning parameters are used for the consecutive batches as for the initial batch mentioned above. In Continue-SGD, in order to preserve some information from the initial batch, we retrain only the last layer and Block-4 of Resnet for the subsequent batches, and do so for 2 epochs only.

For the implementation of iCaRL, we use the code provided by the authors. For each incremental batch, the network is trained for 10 epochs. We start with a learning rate of 0.001 and divide it by 2 after every 2 epochs. Exemplar size is fixed at 5000, to match the protoset size of 100 samples per class for the proposed method.

D. Results

Table I compares the classification accuracies of our proposed method with the other methods, starting from the initial batch and incorporating new object classes incrementally. Cumulative retraining has the highest accuracy among the reported methods, but it is not incremental in nature. The naive method of Continue-SGD evidently suffers from catastrophic forgetting and has very low classification accuracies. Among the incremental methods, iCaRL has the best performance. Accuracies for the proposed method are only slightly lower ($< 2\%$) than iCaRL. A visual comparison of accuracies has also been presented in Fig-3 (left).

Fig-3 (right) compares the training times of the algorithms. The proposed method takes less than 200 seconds to train on an incremental batch of 5 new objects ($\sim$12000 training instances). This is on the same order of magnitude as NCM which requires around 80 seconds, but has lower accuracy as reported in Table I. On the contrary, iCaRL, when trained for 10 epochs requires around 4000 seconds per incremental batch, which is at least 20x more than the proposed method. We also noted that if iCaRL is trained for fewer epochs, the accuracy drops. Finally, cumulative retraining requires very high training times and hence is not feasible at all.

V. Conclusion

This paper presents HiFI, a hierarchical framework for object classification that can be trained in an incremental fashion. The key features of HiFI are: *(i)* it uses a fixed deep feature based representation that does not require retraining for new batches of objects, *(ii)* it adopts a hierarchy of classifiers so that when new objects are presented, only the relevant classifiers can be independently retrained without having to retrain the entire model, and *(iii)* it maintains a protoset of training samples for each encountered class to facilitate incremental training. Experimental results on the CORe50 dataset lead to conclusion that HiFI can achieve classification accuracies comparable to the state-of-the-art iCaRL method, while requiring 20x lesser time than iCaRL for training on incremental batches of data.

In the current work, we have assumed that the initial training batch comprises of at least one object belonging to each category. It would be fairly straight-forward to handle the case of a new category by adding new protoset vectors for the new category in the first stage, and training a new Stage-2 classifier.

As part of the future work, we plan to do the following: (1) Develop a method to automatically discover the hierarchical structure in a given dataset. For example, one could use k-means clustering to group the object classes into different clusters, and use the resulting labels to define categories for Stage-1 classification, thereby, avoiding the human effort needed for generating category labels for various products. (2) Extend the ideas developed in this paper to the much less controlled and more challenging e-commerce retail warehouse setting where one has to deal with millions of products, thereby, solving the extreme classification problem.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in neural information processing systems*, 2015, pp. 91–99.

[3] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *IEEE International Conference on Computer Vision (ICCV), 2017*. IEEE, 2017, pp. 2980–2988.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.

[5] C. Rashtchian, P. Young, M. Hodosh, and J. Hockenmaier, "Collecting image annotations using amazon's mechanical turk," in *Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk*. Association for Computational Linguistics, 2010, pp. 139–147.

[6] C. K. Singh, A. Majumder, S. Kumar, and L. Behera, "Deep network based automatic annotation for warehouse automation," in *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2018, pp. 1–8.

[7] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.

TABLE I

INCREMENTAL CLASS ACCURACY ON CORE50 DATASET FOR DIFFERENT LEARNING METHODS. CUMULATIVE RETRAINING HAS THE HIGHEST ACCURACY, BUT IS NOT INCREMENTAL IN NATURE. ICARL[8] HAS THE HIGHEST ACCURACY AMONG THE INCREMENTAL METHODS. HIFI HAS ONLY SLIGHTLY LOWER ACCURACY THAN ICARL, BUT SIGNIFICANTLY OUTPERFORMS IN TERMS OF TRAINING TIMES, AS SHOWN IN FIG-3.

Batch	Number of classes	Cumulative retraining	Continue-SGD	NCM	iCaRL	HiFI (proposed)
1	10	98.41	98.41	97.32	96.32	98.33
2	15	98.66	32.94	90.34	91.36	91.01
3	20	95.96	24.85	87.27	89.05	88.52
4	25	94.44	19.86	83.17	86.00	85.82
5	30	95.04	16.37	83.79	87.04	86.44
6	35	92.83	13.94	80.88	84.57	83.64
7	40	92.96	12.30	80.32	83.75	82.85
8	45	93.08	10.80	78.51	82.80	81.56
9	50	92.87	9.97	78.51	83.03	81.15

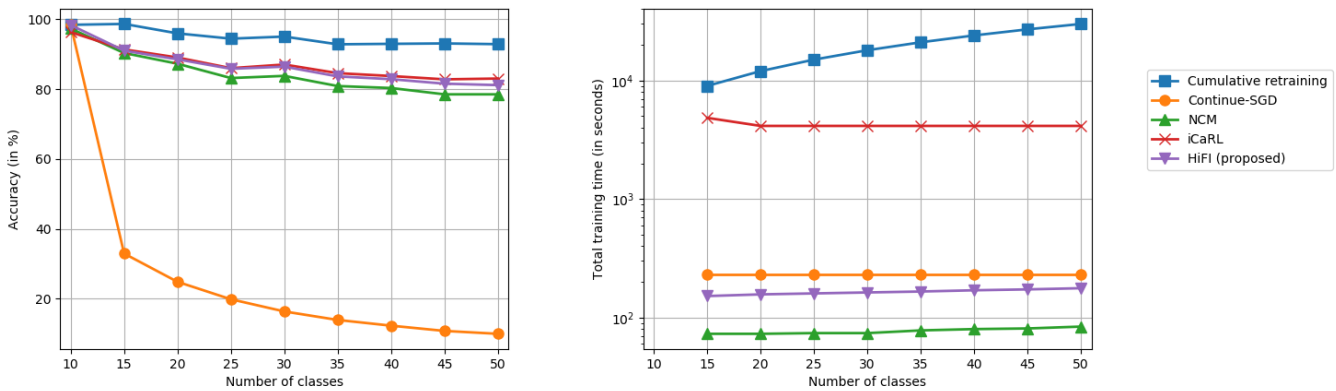

Fig. 3. Accuracies (left) and training times (right) of different methods for incremental batches of CORe50 dataset. Each incremental batch contains 12000 images belonging to 5 new object classes. Training time for the initial batch has not been reported since it can be trained offline without any timing constraints. The proposed method, HiFI, can be trained 20 times faster than iCaRL. The only method faster than HiFI is NCM, but it has lower accuracy.

[8] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. CVPR*, 2017.

[9] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, "Progressive neural networks," *CoRR*, vol. abs/1606.04671, 2016.

[10] D. Lopez-Paz *et al.*, "Gradient episodic memory for continual learning," in *Advances in Neural Information Processing Systems*, 2017, pp. 6467–6476.

[11] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," 2018.

[12] H. Peng, J. Li, Y. Song, and Y. Liu, "Incrementally learning the hierarchical softmax function for neural language models." in *AAAI*, 2017, pp. 3267–3273.

[13] Z. Wu and S. Saito, "HiNet: Hierarchical Classification with Neural Network," *arXiv preprint arXiv:1705.11105*, 2017.

[14] M. Ristin, M. Guillaumin, J. Gall, and L. Van Gool, "Incremental learning of random forests for large-scale image classification," *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 3, pp. 490–503, 2016.

[15] M. Pratama, G. Zhang, M. J. Er, and S. Anavatti, "An incremental type-2 meta-cognitive extreme learning machine," *IEEE transactions on cybernetics*, vol. 47, no. 2, pp. 339–353, 2017.

[16] A. Nichol and J. Schulman, "Reptile: a scalable metalearning algorithm," *arXiv preprint arXiv:1803.02999*, 2018.

[17] M. Oquab, L. Bottou, I. Laptev, and J. Sivic, "Learning and transferring mid-level image representations using convolutional neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 1717–1724.

[18] X. Li, W. Mao, and W. Jiang, "Extreme learning machine based transfer learning for data classification," *Neurocomputing*, vol. 174, pp. 203–210, 2016.

[19] T. Mensink, J. Verbeek, G. Csurka, and F. Perronnin, "Metric learning for nearest class mean classifiers," May 12 2015, uS Patent 9,031,331.

[20] V. Lomonaco and D. Maltoni, "Core50: a new dataset and benchmark for continuous object recognition," *arXiv preprint arXiv:1705.03550*, 2017.

[21] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *arXiv preprint arXiv:1802.07569*, 2018.

[22] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, p. 201611835, 2017.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY DELHI. Downloaded on February 01,2021 at 11:53:15 UTC from IEEE Xplore. Restrictions apply.